When agents go wrong

Browser Use: How LLM Agents Drive the Web

Week 8 Lecture 1

What we will cover

1. The three failure modes: loops, stalls, and errors
2. Loop detection: repeated-action and stagnant-page patterns
3. `max_steps` as the hard ceiling
4. `max_failures` and the error cascade problem
5. `final_response_after_failure`: getting a structured output from a failing run
6. Reading failures from the agent history

Three ways an agent fails

The three failure modes

Looping: the agent takes the same action repeatedly on a page that does not change

Stalling: the agent tries many different actions but the page state stays the same

Erroring: an action throws an exception; the `ActionResult` carries an error field

Each mode looks different in the history.
Each mode has a different fix.

Why looping happens

- Agent clicks a button. Page reloads to the same state.
- `evaluation_previous_goal` reads "Success" even though nothing changed.
- Agent decides to click the same button again.
- Without detection: all remaining steps consumed in this cycle.

The model has no persistent memory of the DOM state outside what `MessageManager` assembles.
Loop detection adds that awareness from outside the model.

Loop detection

What loop detection watches

```
Agent(
    task=task,
    llm=llm,
    loop_detection_enabled=True,    # default: on
    loop_detection_window=20,       # steps in the sliding window
)
```

Two patterns monitored over the window:

1. **Repeated-action loop:** same action + same parameters, more than once
2. **Stagnant-page loop:** DOM snapshot and URL unchanged across consecutive steps

When detected: the framework injects a signal into the next step's context so the model can adapt.

Turning detection off: the cost

If `loop_detection_enabled=False`:

- A task that reaches an impossible page state consumes every remaining step before stopping
- You pay for every token in those wasted steps
- You get no diagnostic signal at the detection point

The window size (default 20) controls sensitivity.
Smaller window catches loops faster but risks false positives on legitimately repetitive tasks (e.g., paginating search results).

Step and failure limits

max_steps: the hard ceiling

```
agent = Agent(
    task=task,
    llm=llm,
    max_steps=50,   # set this explicitly
)
```

- When the step counter hits `max_steps`, the loop ends.
- Default varies by environment: always set it explicitly.

Estimating the right value:

Count the discrete page interactions the task needs.

Multiply by 2-3 to account for retries and navigation errors.

That is your starting point.

max_failures: stopping error cascades

```
agent = Agent(
    task=task,
    llm=llm,
    max_steps=40,
    max_failures=2,   # default: 3
)
```

- Counts **consecutive** failed steps, not total failures
- Resets to zero on any successful step
- When the counter hits `max_failures`, the loop ends immediately

Why this matters: an agent that fails three times in a row is unlikely to recover without intervention. Continuing wastes tokens and produces no useful output.

Forced done and reading the history

final_response_after_failure

```
Agent(
    task=task,
    llm=llm,
    max_failures=3,
    final_response_after_failure=True,   # default: on
)
```

When the agent hits `max_failures` or `max_steps`:

1. One final LLM call is made
2. The action set is constrained to `done` only
3. The agent sees the full history and produces a structured `done` action
4. The `done` text contains the agent's self-assessment of what went wrong

Result: a structured output from every run, even failed ones.

Reading failures from the history

```python
for i, item in enumerate(history):
    brain = item.model_output.current_state
    print(f"Step {i+1}")
    print(f"  eval  : {brain.evaluation_previous_goal}")
    print(f"  memory: {brain.memory}")
    print(f"  goal  : {brain.next_goal}")
    for r in item.result:
        if r.error:
            print(f"  ERROR : {r.error}")
```

Diagnostic signals to look for:

- `evaluation_previous_goal` says "Success" but the page did not change: loop pattern
- Same `next_goal` across many steps: stall pattern
- Non-empty `error` fields: error cascade

The four-step reliability pattern

1. **Cap:** set `max_steps` and `max_failures` explicitly
2. **Detect:** leave `loop_detection_enabled=True`
3. **Force:** keep `final_response_after_failure=True`
4. **Diagnose:** read the history after every run that does not succeed

No single setting fixes a failing agent.
All four mechanisms work together.

> "An agent is just a for-loop of messages."

browser-use team (Gregor Žunič), The Bitter Lesson of Agent Frameworks, 2026

What is next

Lecture 2 (this week): planning across steps, procedural memory, and task framing as a reliability lever

Section (this week): take a flaky agent, read its history, diagnose the failure mode, and make it pass by reframing and adding guards

Reading: week 8 reading covers all of today's mechanisms plus the planner and memory field

Takeaways

Takeaways

- Agents fail as loops, stalls, or errors. Each mode needs a different fix.
- Loop detection watches a sliding window. Leave it on.
- `max_steps` is the hard ceiling. Set it explicitly based on task complexity.
- `max_failures` stops consecutive error cascades. The counter resets on success.
- `final_response_after_failure=True` gives you a structured `done` action even from a failed run.
- The history is your diagnostic instrument. Read it after every failed run.